Problem 4.A Broken Keyboard

You're typing a long text with a broken keyboard. Well it's not so badly broken. The only problem with the keyboard is that sometimes the "home" key or the "end" key gets automatically pressed (internally).

You're not aware of this issue, since you're focusing on the text and did not even turn on the monitor! After you finished typing, you can see a text on the screen (if you turn on the monitor).

In Chinese, we can call it Beiju. Your task is to find the Beiju text.

Input

There are several test cases. Each test case is a single line containing at least one and at most 100,000 letters, underscores and two special characters '`[`' and '`]`'. '`[`' means the "Home" key is pressed internally, and '`]`' means the "End" key is pressed internally. The input is terminated by end-of-file (EOF).

Output

For each case, print the Beiju text on the screen.

Sample Input

```
This_is_a_[Beiju]_text
[[]][][]Happy_Birthday_to_Tsinghua_University
```

Sample Output

```
BeijuThis_is_a__text
Happy_Birthday_to_Tsinghua_University
```

Problem 4.B File Fragmentation

Your friend, a biochemistry major, tripped while carrying a tray of computer files through the lab. All of the files fell to the ground and broke. Your friend picked up all the file fragments and called you to ask for help putting them back together again.

Fortunately, all of the files on the tray were identical, all of them broke into exactly two fragments, and all of the file fragments were found. Unfortunately, the files didn't all break in the same place, and the fragments were completely mixed up by their fall to the floor.

You've translated the original binary fragments into strings of ASCII 1's and 0's, and you're planning to write a program to determine the bit pattern the files contained.

Input

?The input begins with a single positive integer on a line by itself indicating the number of the cases following, each of them as described below. This line is followed by a blank line, and there is also a blank line between two consecutive inputs.

Input will consist of a sequence of "file fragments", one per line, terminated by the end-of-file marker. Each fragment consists of a string of ASCII 1's and 0's.

Output

For each test case, the output must follow the description below. The outputs of two consecutive cases will be separated by a blank line.

Output is a single line of ASCII 1's and 0's giving the bit pattern of the original files. If there are $2N$ fragments in the input, it should be possible to concatenate these fragments together in pairs to make N copies of the output string. If there is no unique solution, any of the possible solutions may be output.

Your friend is certain that there were no more than 144 files on the tray, and that the files were all less than 256 bytes in size.

Sample Input

```
1

011
0111
01110
111
0111
10111
```

Sample Output

```
01110111
```

Problem 4.C Abstract Names

Some of you may have noticed that in certain computer games, particularly the ones based on sports, the spelling of names are mutated so that they are not an exact duplicate of the real entity. This is done to avoid hassles of taking permission from each player as well as any patent issues. In this problem, you will be given a pair of names, one of which is that of a player in real life and the second found in a game. You will have to determine if the two names are same, that is the second one is obtained by mutating the first.

Two names are considered same if they are of same length and they only vary at positions where vowels occur. That means, a name which can be obtained by replacing zero or more vowels by other vowels to obtain a new name are considered same, provided they have same length. For example, both *polo* and *pola* are same as *pele* but not *pelet* or *bele*.

Input

First line of input contains a positive integer $n \leq 20$, where n denotes the number of test cases. This will be followed by $2 * n$ lines where each line will contain a name of at most 20 characters. The names will consist of lower case letters only.

Output

For each case of input, there will be one line of output. It will be '`Yes`' if the second name can be obtained by mutating the first name, otherwise it will be '`No`'.

Sample Input

```
5
pele
polo
pele
pola
ronaldo
ronaldino
pele
pelet
pele
bele
```

Sample Output

```
Yes
Yes
No
No
No
```

Problem 4.D Train Swapping

At an old railway station, you may still encounter one of the last remaining "train swappers". A train swapper is an employee of the railroad, whose sole job it is to rearrange the carriages of trains.

Once the carriages are arranged in the optimal order, all the train driver has to do, is drop the carriages off, one by one, at the stations for which the load is meant.

The title "train swapper" stems from the first person who performed this task, at a station close to a railway bridge. Instead of opening up vertically, the bridge rotated around a pillar in the center of the river. After rotating the bridge 90 degrees, boats could pass left or right.

The first train swapper had discovered that the bridge could be operated with at most two carriages on it. By rotating the bridge 180 degrees, the carriages switched place, allowing him to rearrange the carriages (as a side effect, the carriages then faced the opposite direction, but train carriages can move either way, so who cares).

Now that almost all train swappers have died out, the railway company would like to automate their operation. Part of the program to be developed, is a routine which decides for a given train the least number of swaps of two adjacent carriages necessary to order the train. Your assignment is to create that routine.

Input

The input contains on the first line the number of test cases (N). Each test case consists of two input lines. The first line of a test case contains an integer L, determining the length of the train ($0 \leq L \leq 50$). The second line of a test case contains a permutation of the numbers 1 through L, indicating the current order of the carriages. The carriages should be ordered such that carriage 1 comes first, then 2, etc. with carriage L coming last.

Output

For each test case output the sentence: '`Optimal train swapping takes` S `swaps.`' where S is an integer.

Sample Input

```
3
3
1 3 2
4
4 3 2 1
2
2 1
```

Sample Output

```
Optimal train swapping takes 1 swaps.
Optimal train swapping takes 6 swaps.
Optimal train swapping takes 1 swaps.
```